

Introduction

Python and R for Data Science

Data Science and Management

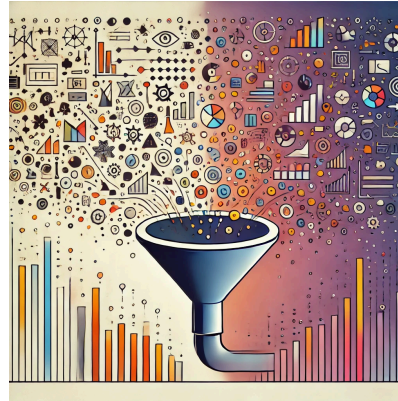


Who we are

- Teacher: Emilio Coppa
 - Position: Assistant Professor @ LUISS
 - Research topics: software testing, vulnerability detection
 - Extra: Team Manager of TeamItaly, the italian cybersecurity team
 - Other info: <https://ecoppa.github.io>
 - Contact: ecoppa@luiss.it
- Teaching Assistant for Python: Lorenzo Ariemma
 - PhD in Computer Science and Automation

Why do we care about *Data Science*?

We can understand a lot of our everyday life by *analyzing* the existing *data* from our world. Such approach is called Data Science:



An interdisciplinary field that uses:

- statistics and processes
- scientific computing and methods
- scientific visualization
- algorithms and systems

to extract or extrapolate knowledge and insights from potentially noisy, structured, or unstructured data.

But, more concretely?

We need to learn a bit of:

- computer programming
- data science frameworks for:
 - data processing
 - data visualization
 - data learning

Why computer programming?

Most everyday tasks require to perform:

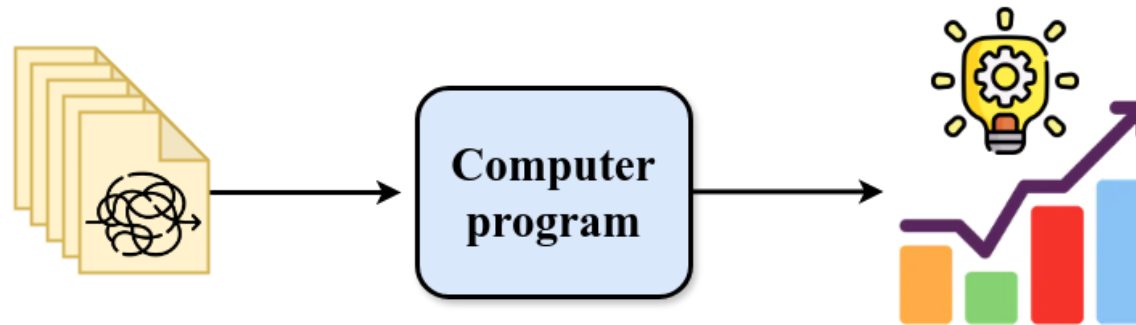
- repetitive...
- long sequences of...
- simple steps

This scenario is *exactly* where a computer excels!

Computers are incredibly fast, accurate, and stupid. Human beings are incredibly slow, inaccurate, and brilliant. Together they are powerful beyond imagination.

-- Albert Einstein

Why do ***you*** need to learn computer programming?



It give us the means to automatically analyze the data and extract knowledge from it.

Solving a task => Algorithm => Recipe followed by a computer

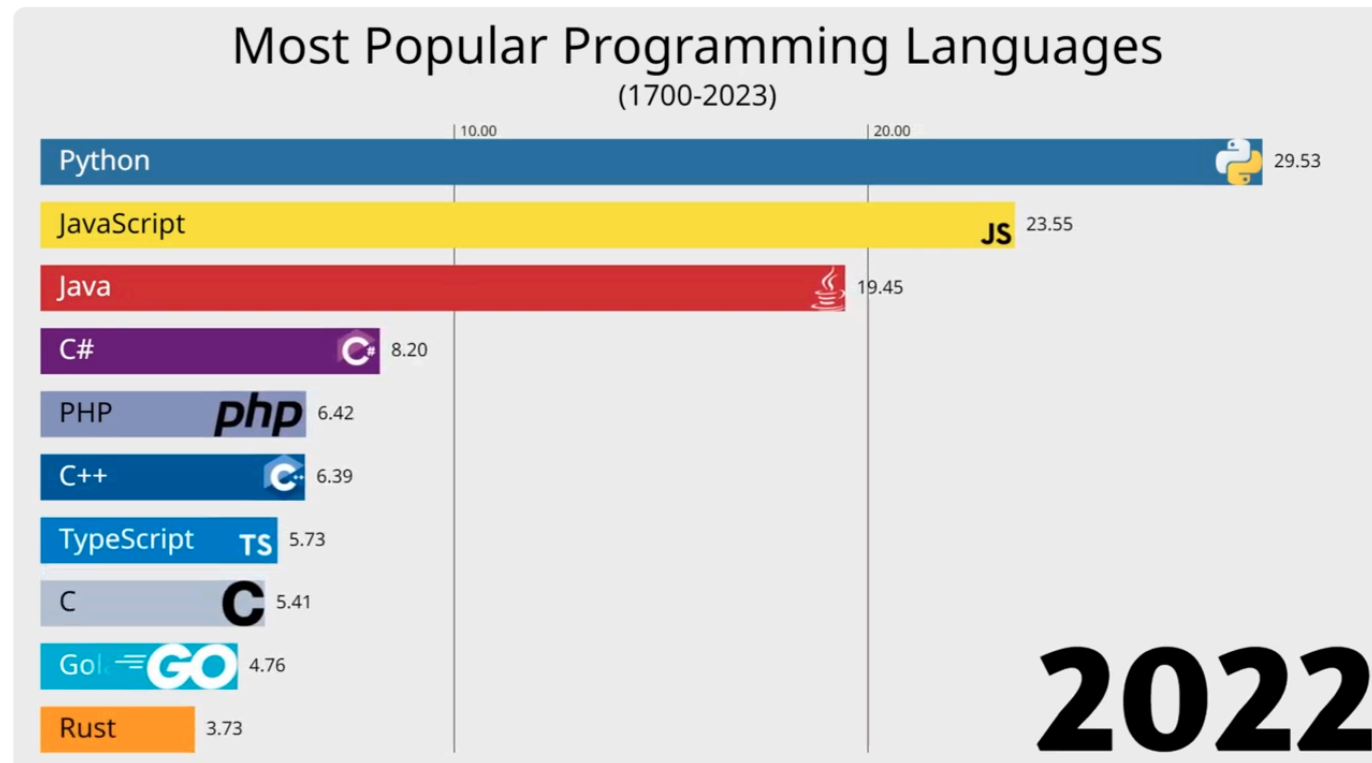
An algorithm features:

- a sequence of simple steps
- flow of control to specify when each step must be executed
- a way of determining when to stop.

...just like a recipe!

However, we cannot write an algorithm in our own natural language (e.g., english). To make an algorithm interpretable by a computer, we need to learn a programming language.

Most Popular Programming Languages



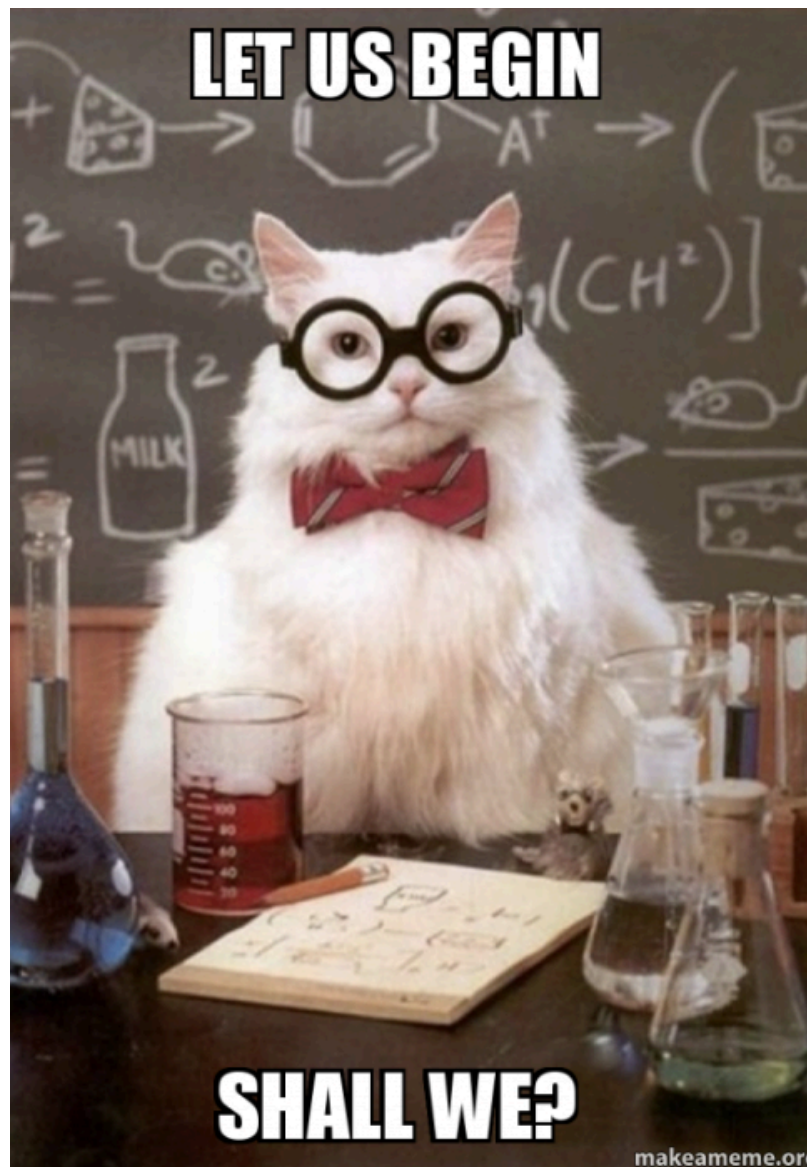
Which Computer Programming Language?



Benefits:

- Easy to learn: designed to be readable, quick to learn
- High-level language: abstracting away most low-level aspects of a computer
- Large community and great support: several libraries allow to build complex projects with just a few lines of code
- Adequate for most tasks: it shows its limits when performance is essential and low-level details play a role. You will hardly work on anything that cannot be implemented in Python

LET US BEGIN



SHALL WE?

makeameme.org

How and Where

How can we run Python code?

Two approaches:

1. Interactive shell:

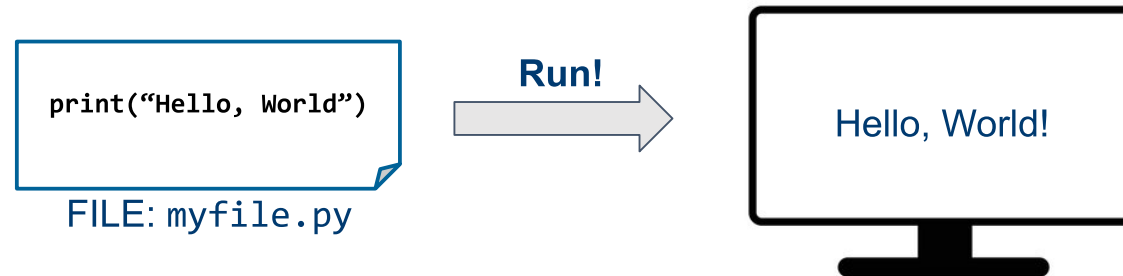
```
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>> █
```

- workflow:
 - start the interactive Python shell
 - write one line of code in the prompt after the symbols `>>>`
 - push enter to execute it and get the results
 - repeat as much as needed
- not very user-friendly
- appropriate only for quick tests
- nothing is saved: you will lose your code when exiting the shell!

How can we run Python code? (cont'd)

Two approaches:

2. Write the code into a textual file and then execute it:



- workflow:
 - write the code into a textual file with extension `.py`
 - execute the file with the Python VM
- This is what you want to do!
- Still *iterative* approach: write the code, check if it works, refine the code, ...

The code can be very cryptic...

Can we add some notes around it?

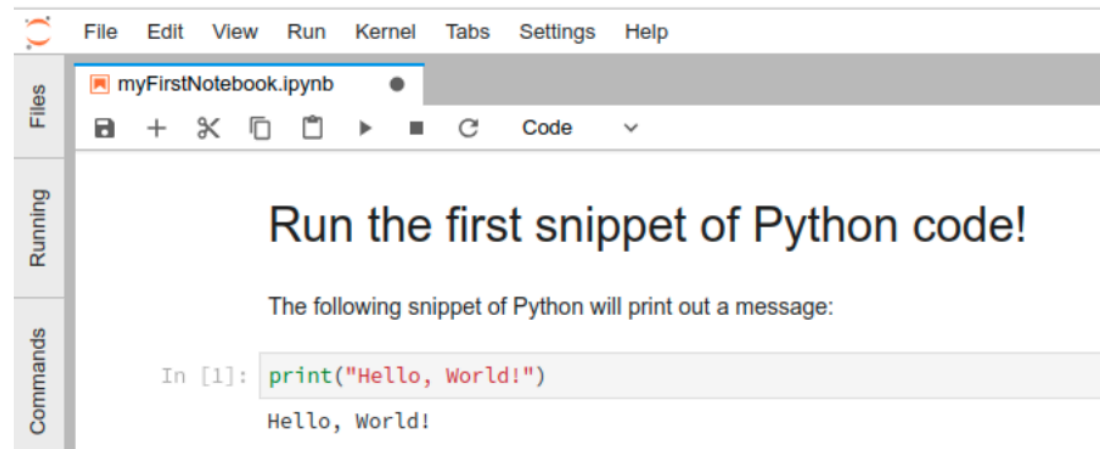
It could be hard to remember why we have written a piece of code, what are its effects and downsides.

It is often convenient to organize our code into "sections" and then add some text to describe what the code is expected to do. This is called:



Jupyter Notebook

Using the environment JupyterLab we can create a new notebook:



Text section
describing what we
is relevant for us
(humans!)

snippet of code

result of the code
(after running it with
the play button)

Where do we run our code/notebooks?

1. Local setup: your computer!



- you have to install Python and all the other bits
- limited by your computational power

2. Cloud setup:



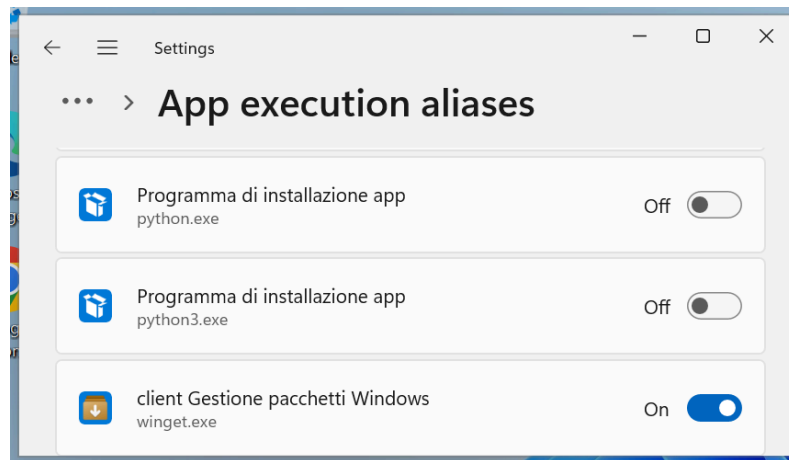
- no need to install everything
- limited by your money

Local Setup

[Only on Windows 11]

Disable Windows Store Aliases

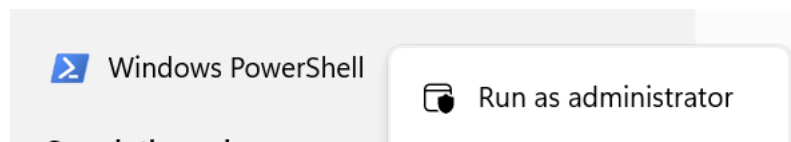
1. From the start menu, search and then open *Manage app execution aliases* (*Alias di esecuzione delle app*)
2. Disables aliases for *python.exe* and *python3.exe*:



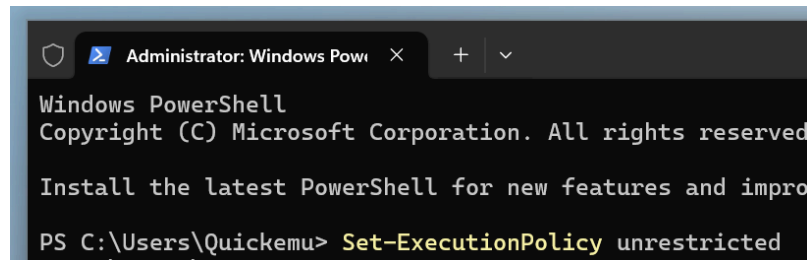
[Only on Windows 11]

Enable script cmdlet

1. From the start menu, search *Windows Powershell*, then using the right-click select *Run as administrator*.

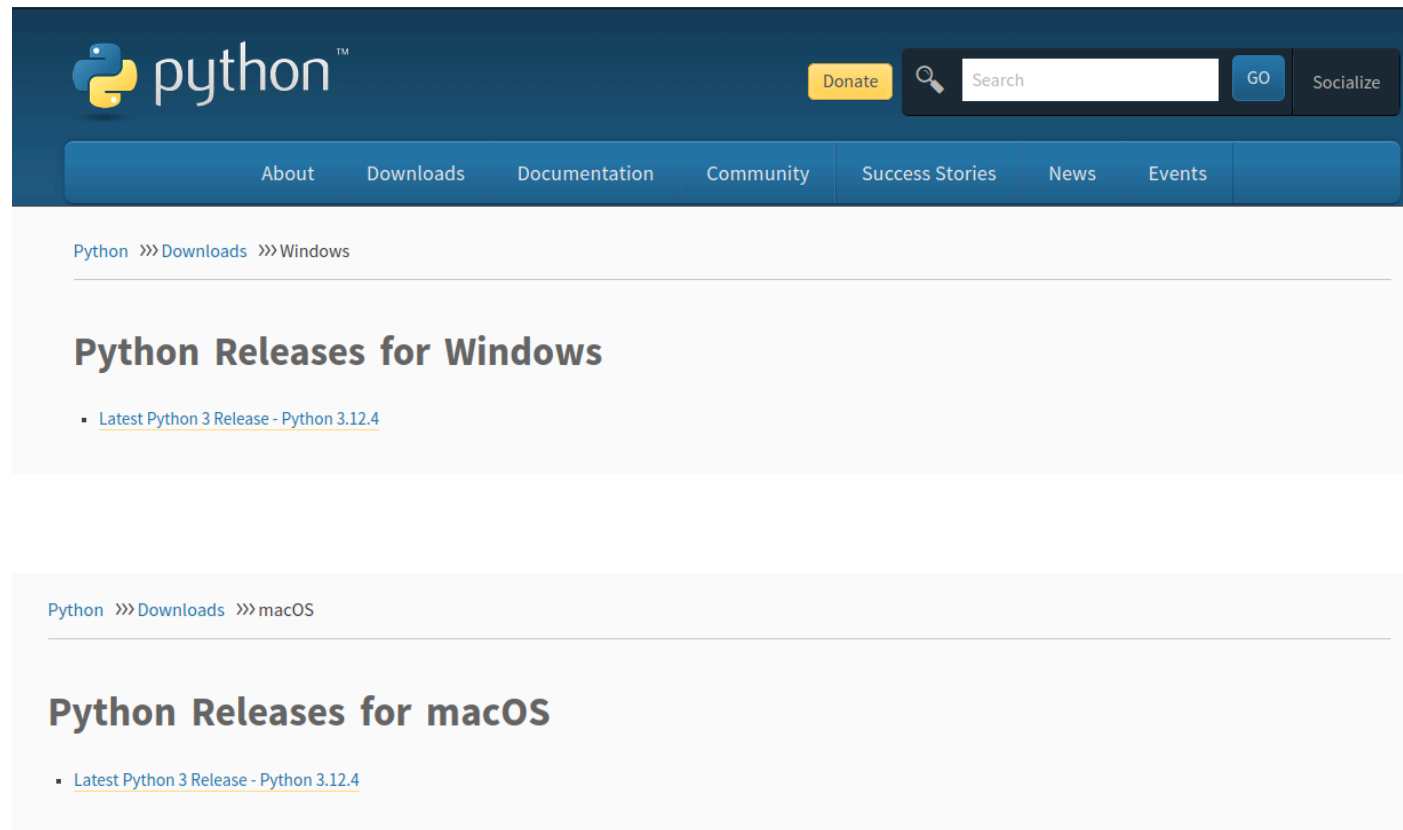


2. Type `Set-ExecutionPolicy unrestricted`, then run it and reply y (or "s" if your Windows is in italian) :



Download Python

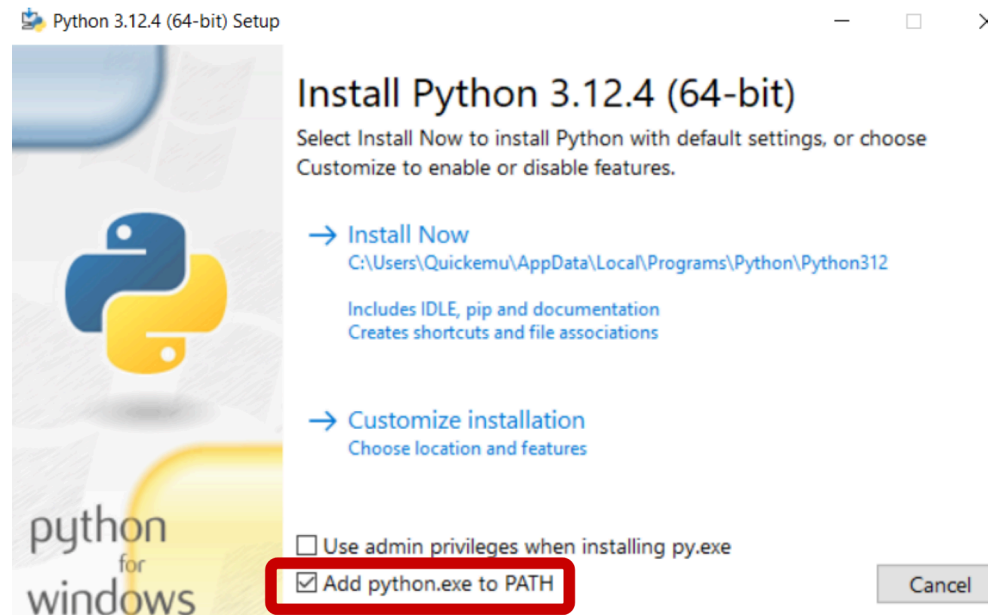
Visit: <https://www.python.org/>



Launch the installer

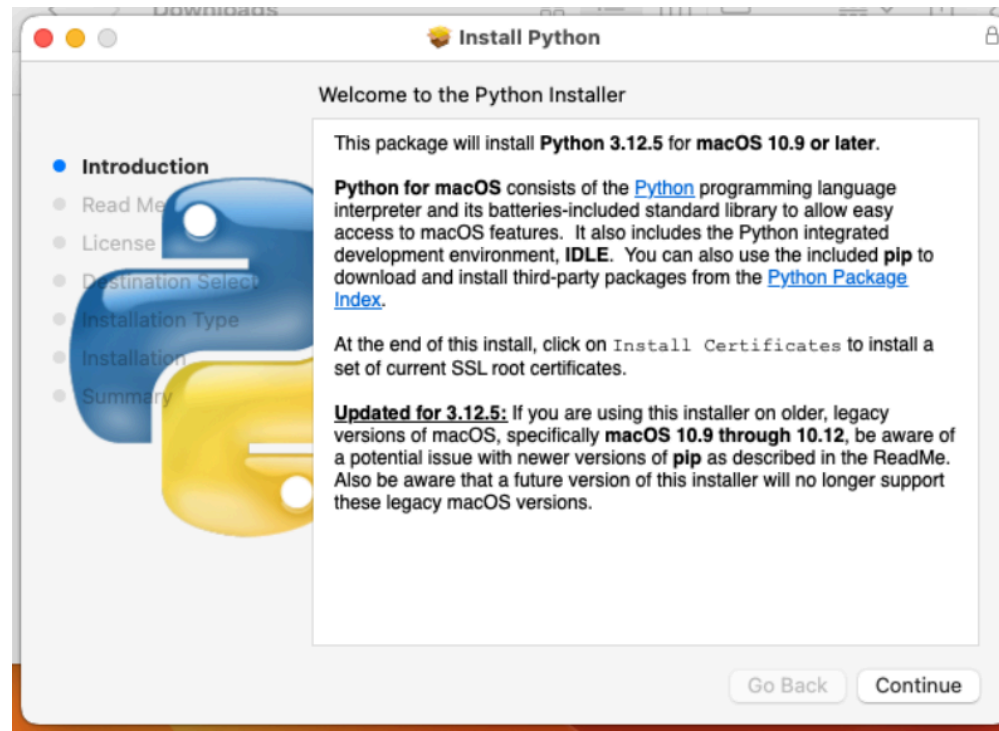
Double click on the downloaded file, e.g., on Windows:

1. Enable ***Add python.exe to PATH***
2. Click *Install now*



Launch the installer (cont'd)

Similarly on Mac OS X:



Install a (good) code editor: **Visual Studio Code**

1. Visit: <https://code.visualstudio.com/Download>
2. Download the latest version for your OS

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.


↓ Windows
Windows 10, 11

User Installer x64 Arm64
System Installer x64 Arm64
.zip x64 Arm64
CLI x64 Arm64

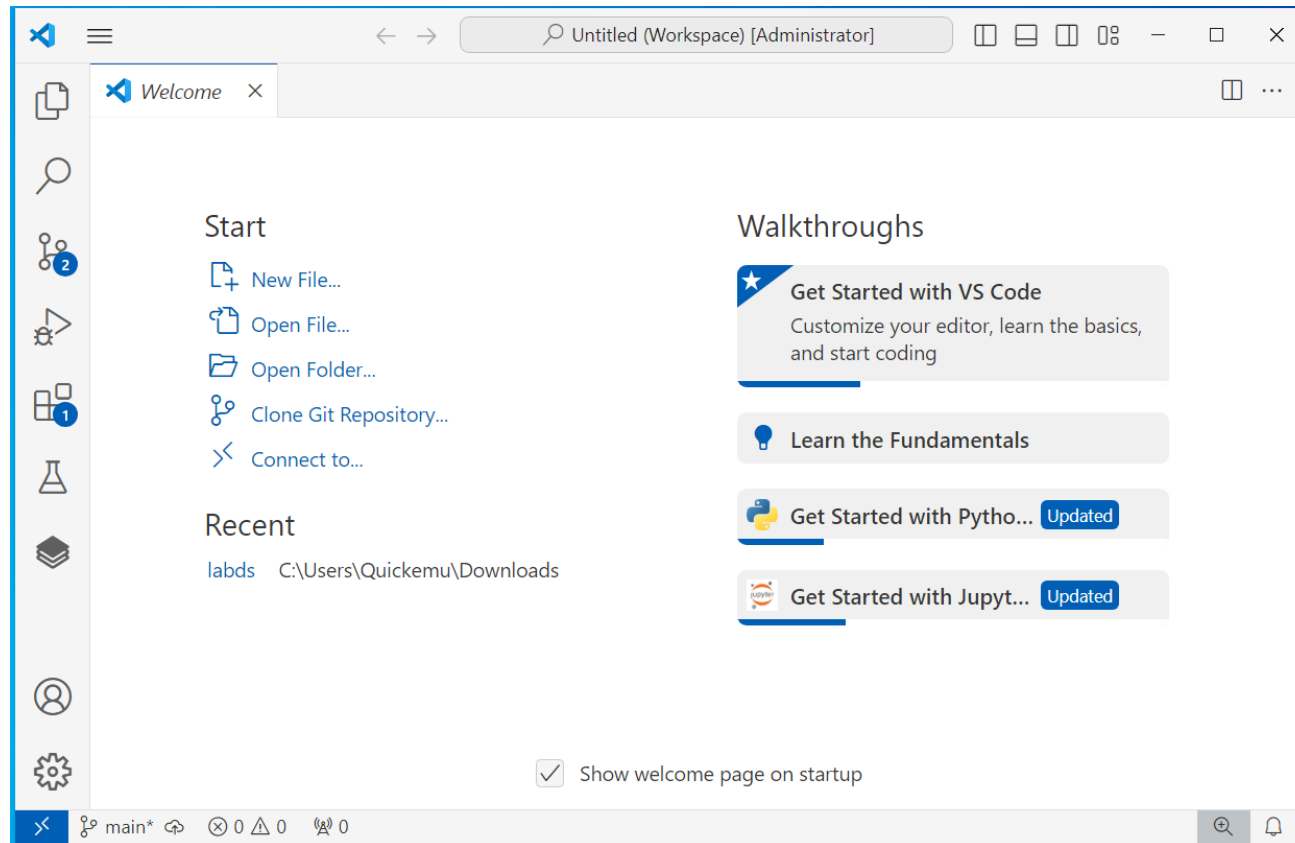

↓ .deb Debian, Ubuntu **↓ .rpm** Red Hat, Fedora, SUSE

.deb x64 Arm32 Arm64
.rpm x64 Arm32 Arm64
.tar.gz x64 Arm32 Arm64
Snap Snap Store
CLI x64 Arm32 Arm64

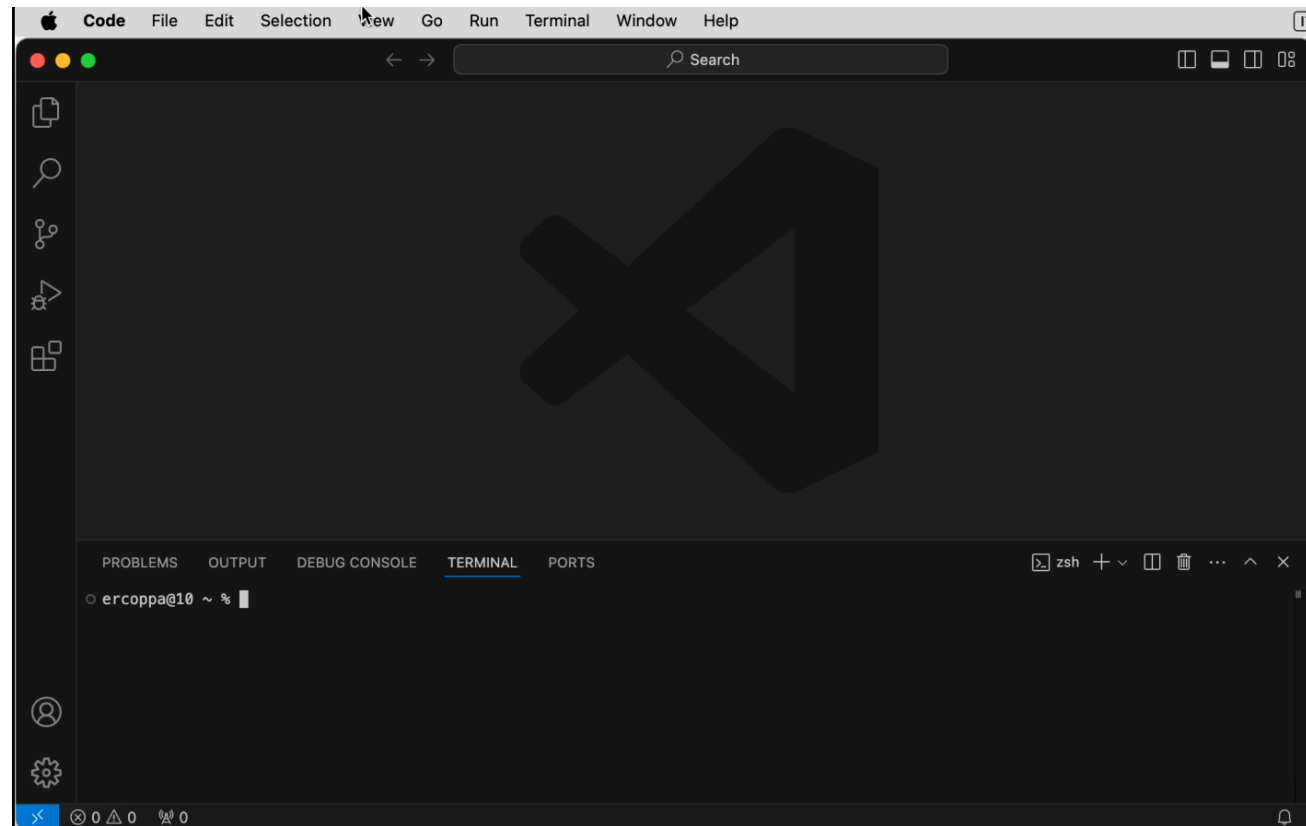

↓ Mac
macOS 10.15+

.zip Intel chip Apple silicon Universal
CLI Intel chip Apple silicon

Start Visual Studio Code (Windows)

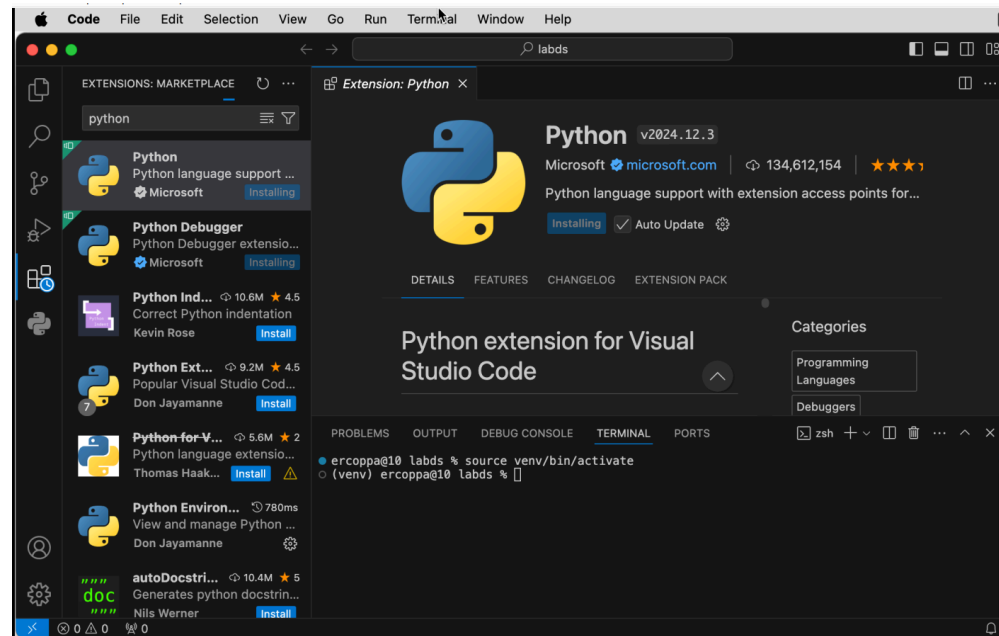


Start **Visual Studio Code** (Mac OS X)



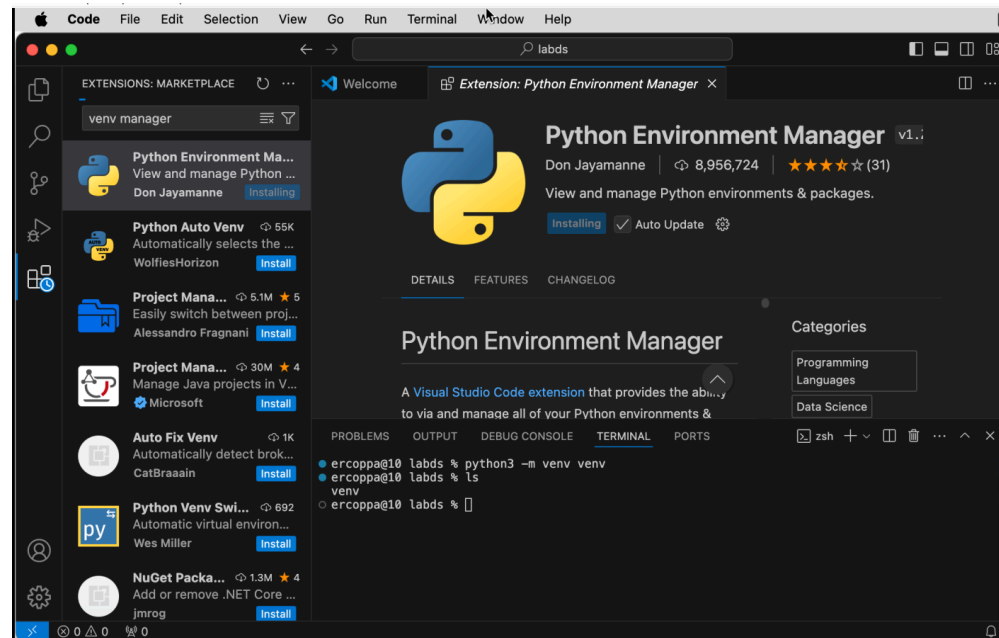
VSCode extension: Python integration

1. From the side bar, open the *Extension* tab
2. Search for *Python*
3. Install the extension



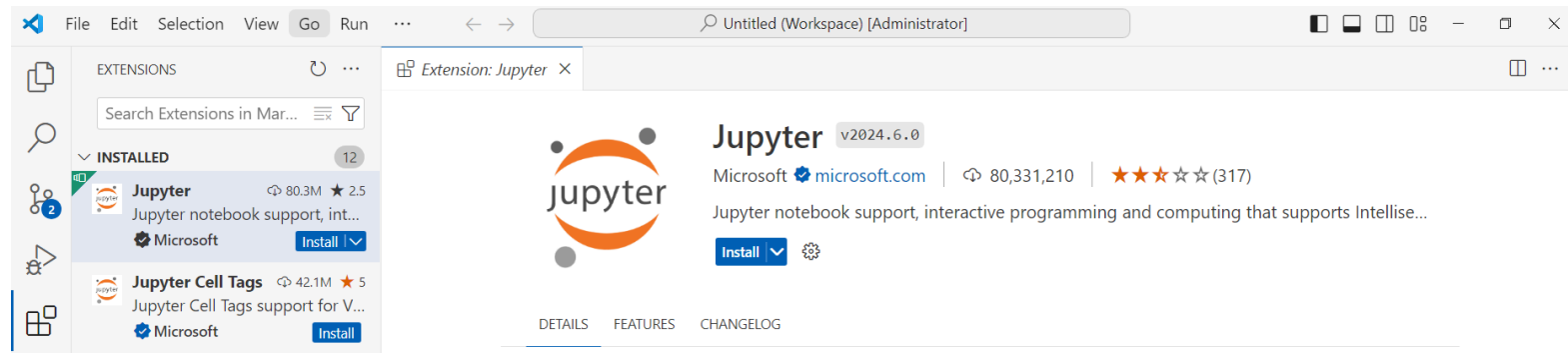
VSCode extension: Python Environment Manager

1. From the side bar, open the *Extension* tab
2. Search for *Python Environment Manager*
3. Install the extension



VSCode extension: Jupyter

1. From the side bar, open the *Extension* tab
2. Search for *Jupyter*
3. Install the extension



Where to install Python packages from the community

We can install Python packages from the community following two strategies:

- *System-wide*: this may create conflicts between different projects using Python.
 - pros: you need to install a package only once for all projects
 - cons: one project may break due to a package installed/updated for another project

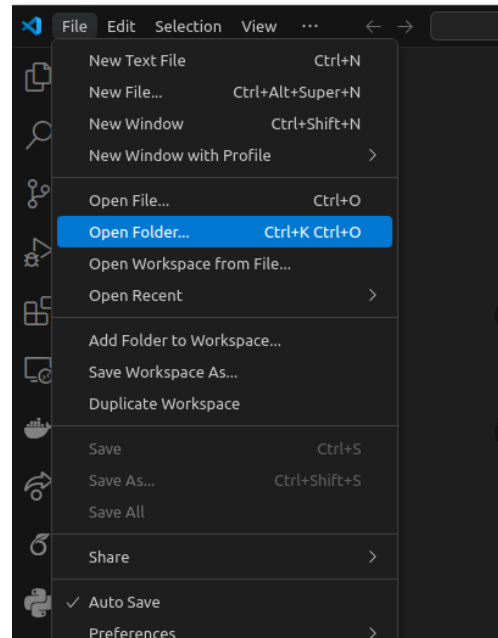
It is now the deprecated approach.

- *Project-wide*: each project has its own *Virtual Environment*.
 - pros: no conflicts across packages of different projects
 - cons: a package must be installed again for each project

This is nowadays the suggested approach and we will follow it.

Create a new virtual environment

1. Create a new folder (e.g., `labds` within `Desktop` folder)
2. Open the folder from VSCode:



3. Start the terminal: *View > Terminal*

4. Type in the terminal:

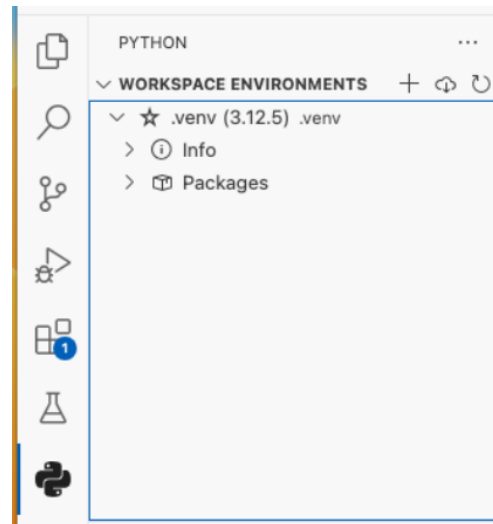
```
python -m venv .venv
```

or:

```
python3 -m venv .venv
```

Pick the default virtual environment

1. From the side bar, open the Python logo
2. Click on the *star* to select the virtual environment



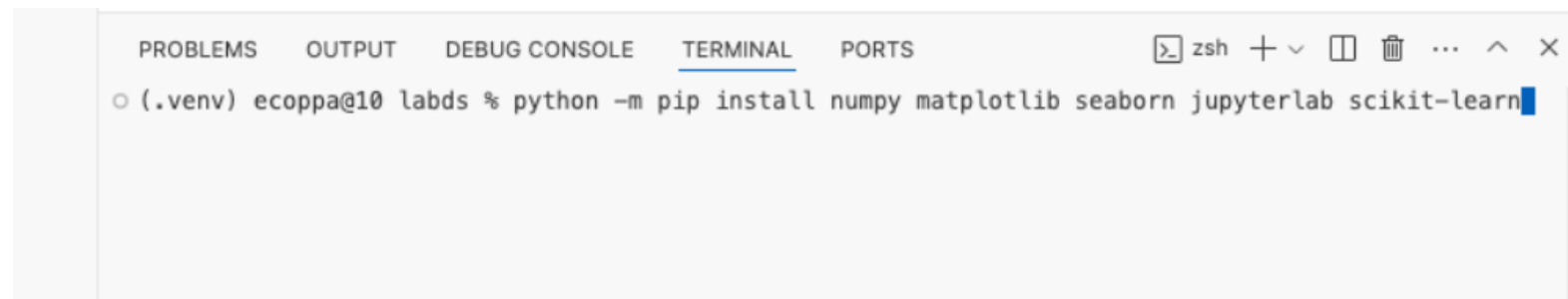
3. Close VSCode and then re-open your project folder in VSCode

Installation of Python packages in the virtual environment

We will use some extra Python bits from the community, including: `numpy`, `matplotlib`, `seaborn`, `scikit-learn`, and `jupyterlab`.

From VSCode:

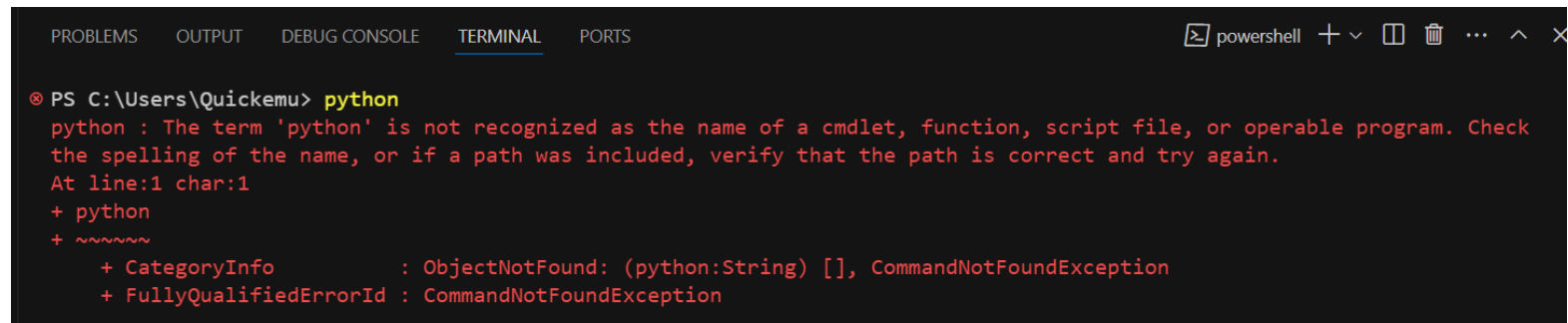
1. Start a new terminal using the menu: *Terminal > New Terminal*
2. `python -m pip install numpy matplotlib seaborn scikit-learn jupyterlab`



The screenshot shows a VS Code interface with a terminal window open. The terminal title bar includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. On the right side of the title bar, there are icons for a terminal, a plus sign, a dropdown arrow, a window icon, a trash icon, an ellipsis, an up arrow, and a close icon. The terminal content shows a prompt `o (.venv) ecoppa@10 labds %` followed by the command `python -m pip install numpy matplotlib seaborn jupyterlab scikit-learn` which has been executed, as indicated by a blue cursor at the end of the line.

Troubleshooting on Windows: python is not recognized

If, when when running `python` in the terminal of VSCode, you get the following error:

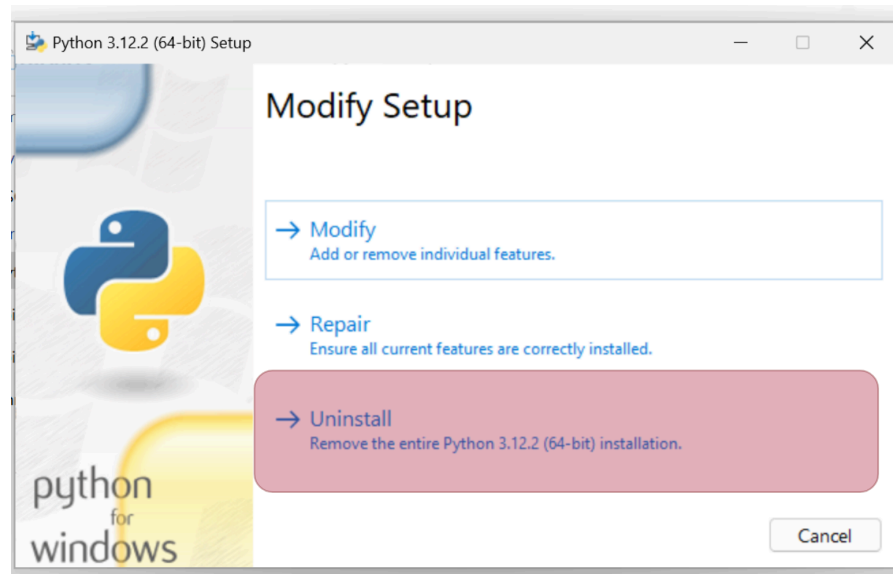


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... ^ x
Ⓢ PS C:\Users\Quickemu> python
python : The term 'python' is not recognized as the name of a cmdlet, function, script file, or operable program. Check
the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ python
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (python:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

Then, you have to:

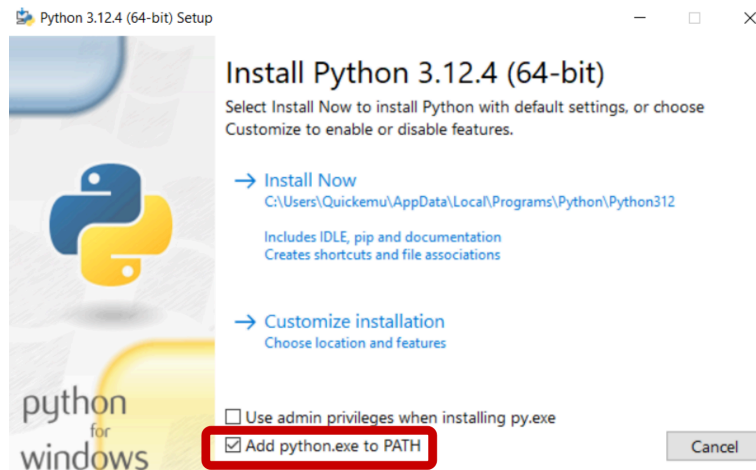
Troubleshooting on Windows: python is not recognized (cont'd)

1. Uninstall Python: run again the installer of Python and select *Uninstall*:



Troubleshooting on Windows: python is not recognized (cont'd)

2. Start the Python installer
3. Select *Add to path* at the bottom



4. Proceed with *Install now*

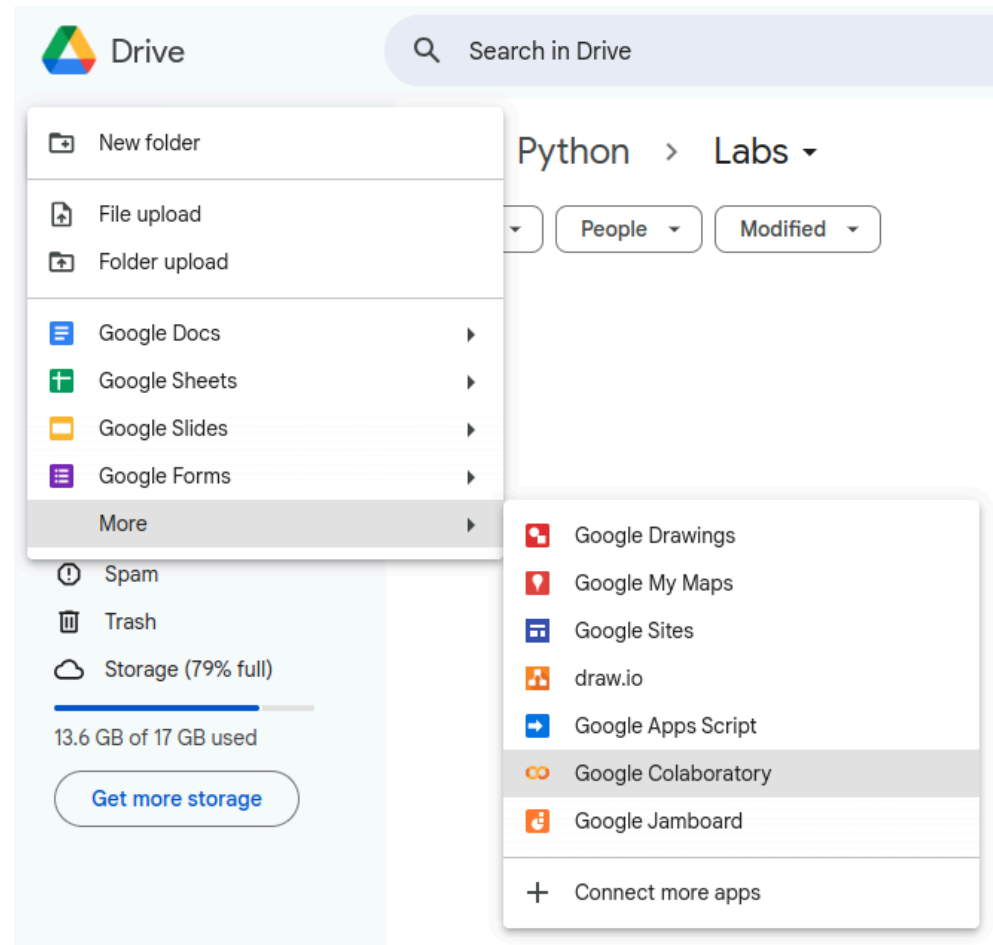
Cloud Setup

Google Colab, i.e., the lazy approach



- Platform from Google
- Google Account is required
- Free of charge to use for the basic plan
- Write code in the browser (no extra is needed)
- Code executed on Google cloud servers
- Low/medium performance, privacy issues
- No need to install anything

From Google Drive



>

Environments

Environment: local setup with VSCode

Use VSCode:

- Quickly write and execute python files (`.py`)
- Quick access to the local terminal
- Write and execute notebooks (`.ipynb`). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Good editor with several extensions (integrations with third-party services)
- Everything runs on your machine

This is the environment that we suggest you to use.

Environment: local setup with JupyterLab

Use JupyterLab:

- Start it from any terminal (e.g., from VSCode) with:
`python -m jupyterlab`
- Write and execute notebooks (`.ipynb`) with support even for advanced and weird features.
- Basic editor
- Everything runs on your machine

We suggest to use JupyterLab only when you need to run advanced Jupyter notebooks obtained from the external world

Environment: cloud setup with Colab

Use Google Colab:

- You can work from any computer. Even your tablet (although, it is a bit inconvenient!)
- Write and execute on the cloud notebooks (.pynb). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Basic editor
- Everything must be saved on Google Drive
- Not adequate in future courses where you need a bit of computational power (assuming you do not want to pay Google).

We suggest to use Colab only if you issues with your local setup

Course Material

What you need

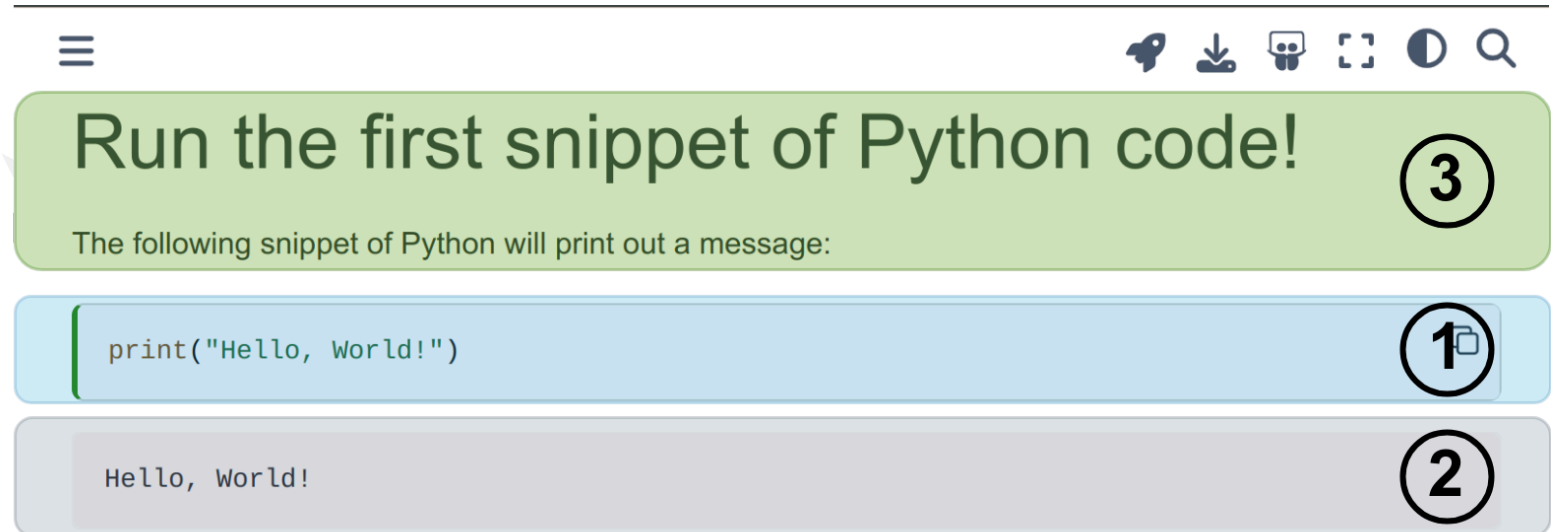
Scan the qrcode to get everything:



<https://ercoppa.github.io/labds>

Each page is a *Notebook*

For instance: <https://ercoppa.github.io/labds/00/00-myFirstNotebook>



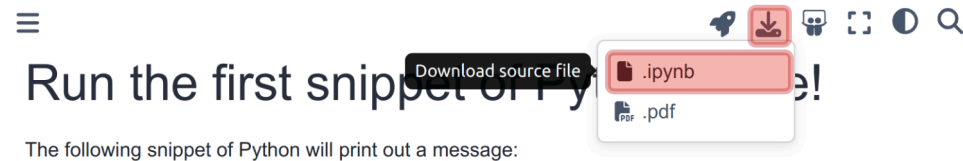
A *notebook* mixes:

1. *code*: something that we can run
2. *result of the code*: we can observe the output of the code execution
3. *text*: used to describe what the code does

You can replay and extend my notebooks!

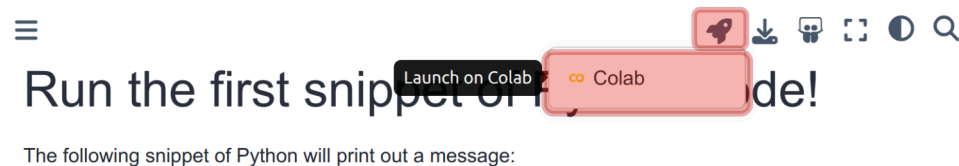
Depending on your setup:

1. Local setup. Run each notebook on your machine. Download the `.ipynb` is easy from the website:



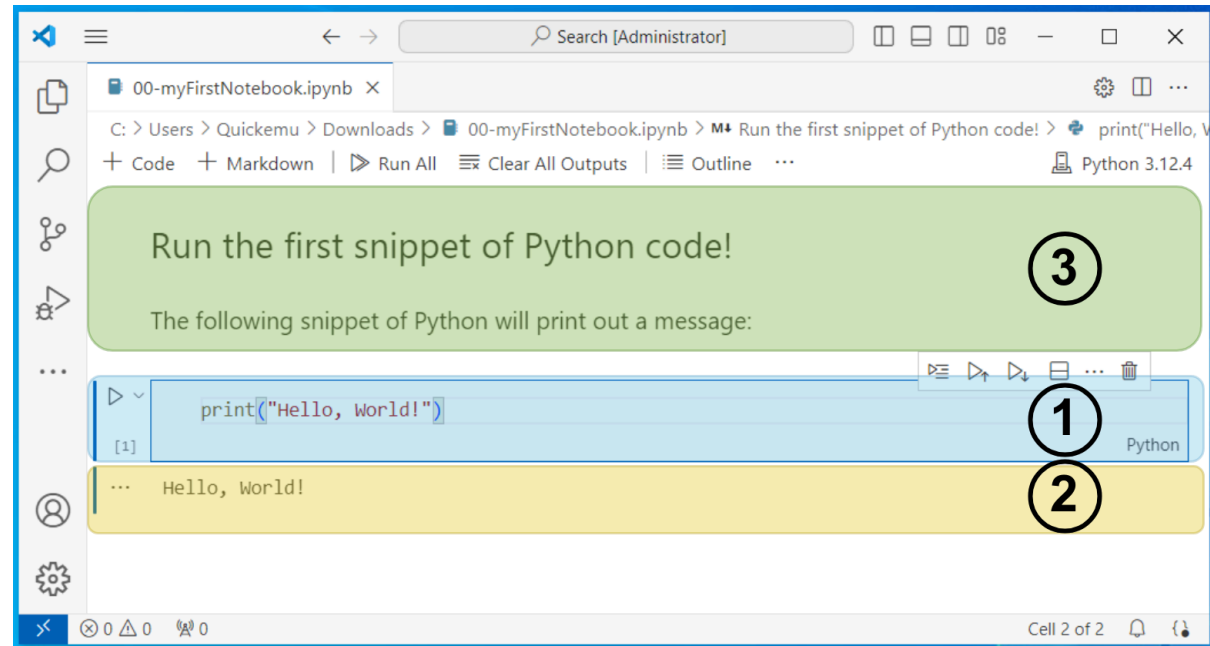
then open it with VSCode or JupyterLab.

2. Cloud setup. Run each notebook on Google Colab (i.e., Google servers). Everything within the browser. To run it on Google Colab:



When opening a notebook on VSCode

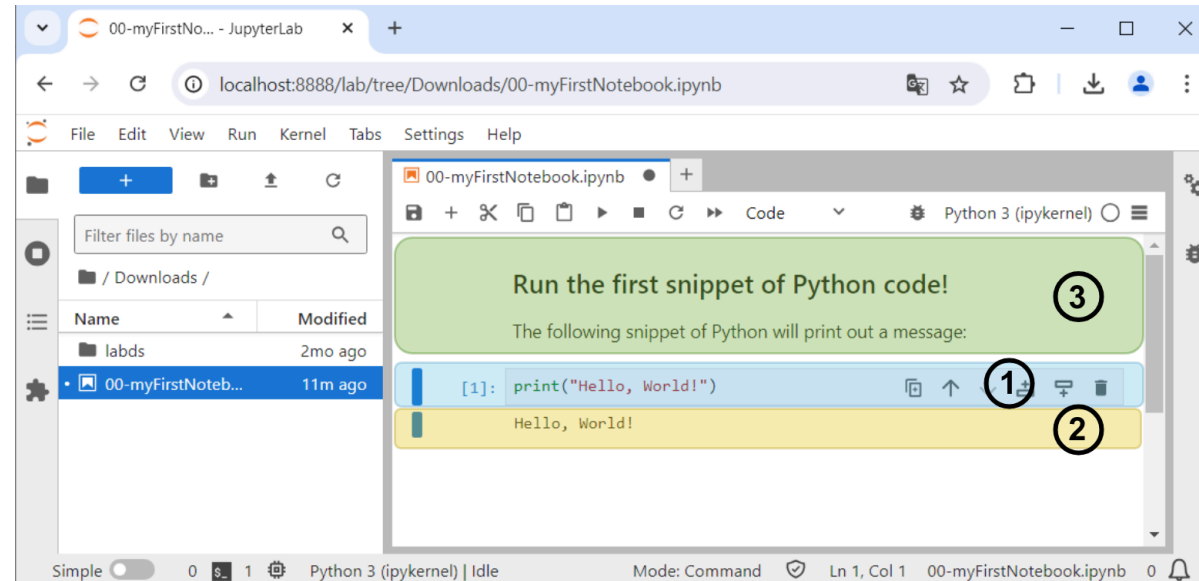
We get the same structure as before:



However, now you can interact with the notebook in several ways.

When opening a notebook on JupyterLab

We get the same structure as before:



However, now you can interact with the notebook in several ways.

When opening a notebook on Colab

We get the same structure as before:

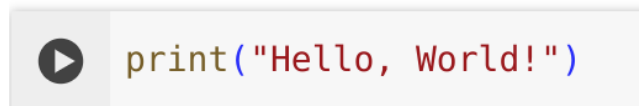


However, now you can interact with the notebook in several ways.

Replaying or updating a notebook

Regardless of the environment (VSCode, JupyterLab, or Colab), we can perform similar interactions with the notebook.

For instance, on Google Colab, if we select a code *cell*, then we get a play button:

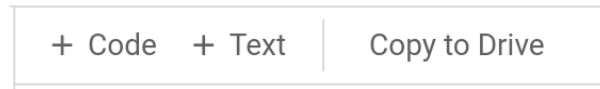


If we push it, we can run the code and (re)generate the execution output:



Moreover, we can modify the notebook:

- double click on existing cells: change their content
- add new code or text cells:



Now, what?

Now, we need to learn a bit of Python!

Check out the page/notebook *Python Basics*.

